

Article

Efficient Autonomous Exploration and Mapping in Unknown Environments

Ao Feng ¹, Yuyang Xie ², Yankang Sun ¹, Xuanzhi Wang ², Bin Jiang ² and Jian Xiao ^{1,*}

¹ College of Integrated Circuit Science and Engineering, Nanjing University of Posts and Telecommunications, Nanjing 210023, China; 1221024723@njupt.edu.cn (A.F.)

² College of Electronic and Optical Engineering & College of Flexible Electronics (Future Technology), Nanjing University of Posts and Telecommunications, Nanjing 210023, China

* Correspondence: xiaoj@njupt.edu.cn

Abstract: Autonomous exploration and mapping in unknown environments is a critical capability for robots. Existing exploration techniques (e.g., heuristic-based and learning-based methods) do not consider the regional legacy issues, i.e., the great impact of smaller unexplored regions on the whole exploration process, which results in a dramatic reduction in their later exploration efficiency. To this end, this paper proposes a Local-and-Global Strategy (LAGS) algorithm that combines a local exploration strategy with a global perception strategy, which considers and solves the regional legacy issues in the autonomous exploration process to improve exploration efficiency. Additionally, we further integrate Gaussian process regression (GPR), Bayesian optimization (BO) sampling, and deep reinforcement learning (DRL) models to efficiently explore unknown environments while ensuring the robot's safety. Extensive experiments show that the proposed method could explore unknown environments with shorter paths, higher efficiencies, and stronger adaptability on different unknown maps with different layouts and sizes.

Keywords: autonomous exploration; perception and mapping; deep reinforcement learning; Gaussian process regression; Bayesian optimization



Citation: Feng, A.; Xie, Y.; Sun, Y.; Wang, X.; Jiang, B.; Xiao, J. Efficient Autonomous Exploration and Mapping in Unknown Environments. *Sensors* **2023**, *23*, 4766. <https://doi.org/10.3390/s23104766>

Academic Editor: Srikanth Saripalli

Received: 4 April 2023

Revised: 9 May 2023

Accepted: 12 May 2023

Published: 15 May 2023



Copyright: © 2023 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

During the autonomous exploration process, robots, without any prior knowledge, rely only on their own sensors and exploration strategies and move through the unknown environment to build a map of the whole circumstance. Autonomous exploration has a wide range of real-life applications, such as the exploration and mapping of unknown environments by rescue robots [1,2], sweeping robots [3,4], and disaster reconnaissance robots [5,6]. In recent years, some remarkable techniques like frontier-based [7,8] and information-based [9,10] methods have emerged. The former method was designed to search for frontier points between the free area and the unknown area and select the optimal frontier point through a global optimization function, which drives robots to the optimal frontier point to build the whole map. The latter utilized Shannon entropy to evaluate the uncertainty of the built map by selecting sensing actions with maximized mutual information (MI), which drives the robot to explore unknown areas. However, due to the complexity and uncertainty of the unknown environment, it is very challenging to formulate a precise and general global optimization function [11]. Therefore, most methods utilize a greedy strategy that minimizes the distance traveled, maximizes the information gained, or both, with fixed weights as the global optimization function without considering future planning. This may lead to a reduction in the efficiency of the robot's exploration and an increase in the total length of exploration paths [12]. In addition, with the richness of map information, each decision process can take an extended amount of time [13], and Deep Reinforcement Learning (DRL) based approaches are considered to solve these problems.

Since DRL gained great success in video games such as Atari [14], some researchers have begun investigating DRL's application in autonomous exploration. Currently, there are two main categories of DRL-based exploration methods. One utilizes raw sensor data (such as depth images [15–17] and LiDAR data [18,19]) as input. This approach is usually an end-to-end approach that drives robot exploration by establishing mapping relationships between raw sensor data and robot control variables. It usually requires consideration of avoiding obstacles in an unknown environment when training, which undoubtedly makes the system more difficult to train [11]. Another type of approach uses environmental features (local environmental features [20–22] or global environmental features [23–26]) as input. This method usually predicts the robot's next direction of motion or target point directly, without considering navigation and obstacle avoidance. Of these, the former uses local features as input and trains robots that are robust to any layout and environment size. However, due to the lack of global information, it often falls into the local optimum problem [27]. The latter uses global features as input which usually has a huge state space, creating a great challenge for the efficient convergence of DRL. As a result, researchers generally supplement the training of the global environment feature method with environment topology [25] and boundary locations [28], but this lowers the adaptability of the system in various environments [12].

Moreover, autonomous exploration methods for unknown environments usually maintain high exploration efficiency in the former stages of exploration while suffering from fairly low exploration efficiency in the later stages of exploration [11]. The main reason for this is the regional legacy issues. Throughout the exploration process, autonomous exploration methods usually pursue unexplored regions with higher gain according to their exploration strategies, resulting in some unexplored regions with less gain being left in various corners of the map. In the latter stages of exploration, the robot takes a long path cost to complete the final exploration. Existing exploration methods usually do not actively address regional legacy issues. Instead, by passively setting an exploration threshold, these methods stop exploration when the exploration ratio reaches the exploration threshold in order to maintain high exploration efficiency [8,29]. However, this directly affects the quality and integrity of the built maps.

To address the common problems of existing exploration methods, such as the region legacy issues and the challenge of balancing the optimal exploration path and environmental robustness with a single exploration strategy, we propose a Local-and-Global Strategy (LAGS) algorithm that integrates a local exploration strategy and a global perception strategy. The local exploration strategy emphasizes high-gain regions in the local map to expedite exploration of unknown areas while ensuring that the algorithm is robust in the environment; the global perception strategy focuses more on maximizing future rewards, guiding the robot to choose the optimal exploration path through the perceived global information to avoid inefficiencies in the later stages of exploration. The LAGS algorithm can proactively solve the regional legacy issues in the autonomous exploration process, increasing the robot's exploration efficiency. In time-sensitive tasks such as human search and rescue and disaster reconnaissance, the LAGS algorithm allows faster exploration of unknown environments to speed up the search for victims. Our contributions are listed below:

1. We analyze the impact of the regional legacy issues on the efficiency of exploration and propose a LAGS algorithm that can solve the regional legacy issues during the exploration process and improve the efficiency of exploration.
2. We combine a local exploration strategy with a global perception strategy, solving the problem that a single exploration strategy is challenging to balance between optimal exploration paths and environmental robustness.
3. We use Gaussian process regression (GPR) and Bayesian optimization (BO) sampling points as candidate action points for the robots. Compared to the classical frontier-based candidate point selection methods, our approach ensures that each candidate action point is safe and has a higher MI gain.

4. Extensive experimental results obtained on various maps with different layouts and sizes show that the proposed method has shorter paths and higher exploration efficiency than other heuristic-based or learning-based methods.

The paper is organized as follows: Section 2 presents related work on the development of autonomous exploration in heuristic-based and learning-based terms; Section 3 describes the system pipeline of our proposed method and the problem definition for the autonomous exploration task; Section 4 describes the GPR prediction of MI reward surfaces and the process of determining candidate action points by BO; Section 5 presents and analyzes the proposed algorithm and network structure; Section 6 conducts a series of simulations and scalability experiments and compares the performance between the different methods; Section 7 summarizes the existing work and discusses possible future research directions.

2. Related Works

The development of autonomous exploration can generally be divided into two categories: the heuristic-based method and the learning-based method. The heuristic-based method is still the most popular method in solving autonomous exploration problems [24], while the learning-based method is considered to be the most promising method and is currently a hot research topic [30].

Among all the heuristic-based methods, frontier-based and information-based algorithms are the most classical. The frontier algorithm was first proposed by Yamauchi [31], who utilized a depth-first search algorithm to drive a robot to the nearest frontier point to complete the exploration of the map. Researchers have proposed various improvements to this method, such as using a rapidly exploring random tree (RRT) algorithm to identify candidate frontier points [32] and also formulating a reasonable and efficient global optimization function to select the optimal frontier point [33,34]. Since the safety of the target points selected based on the frontier algorithm is uncertain, Selin et al. [35] combined the Frontier Exploration Planning (FEP) and Next Best View Planning (NBVP) algorithms to enable the robot to explore more unknown areas as possible while ensuring its safety. In the information-based methods, Bourgault et al. [36] introduced Shannon entropy to evaluate the uncertainty of constructing maps during exploration. Based on this, Ila et al. [37] defined an MI gain and selected paths that minimize map entropy to explore the environment. Julian et al. [38] improved the representation and calculation of MI by proposing a method to calculate MI using a beam-based model. Due to the high cost of calculating MI, Bai et al. [39,40] proposed a method to predict MI based on GPR, where the MI results were only calculated from the training samples of a two-dimensional Sobel sequence. However, this method can only give an approximate prediction of MI. As a result, the BO-based active sampling method was proposed later to improve the prediction accuracy of MI. In general, the results of heuristic-based methods mainly depend on the formulated global optimization function and current observations of the environment without considering future exploration. As a result, longer exploration paths and lower exploration efficiency can always be seen in most heuristic-based methods modeling processes [30].

With the rapid development of deep learning (DL) and reinforcement learning (RL), learning-based methods are considered an effective approach for solving the problems above. DRL, merging DL and RL, has been widely used as an effective sequential decision-making tool in areas such as unmanned aerial vehicle (UAV) navigation [41], multi-objective path planning [42], and computer games [43]. Hence, researchers have considered defining autonomous exploration tasks as a sequential decision problem and using DRL to solve it. Juliá et al. [44] modeled autonomous exploration as a partial observation Markov decision process (POMDP) and used direct policy search to solve it. Tai and Liu [45] used a convolutional neural network (CNN) to extract input image features and trained a deep Q-network (DQN) based control network. However, this algorithm only achieved collision-free roaming in unknown environments. Bai et al. [27] constructed a supervised learning problem to predict the next direction of movement by feeding local features of an environmental map into a neural network. However, this algorithm can only perceive local information about

the map and often falls into local solutions. Thus, Chen et al. [12] trained a recurrent neural network (RNN) and added a Frontier Rescue (FR) strategy to solve the local solution, which still suffers from path redundancy during the exploration process. Dooraki and Lee [46] trained decision networks using deep Q-learning neural networks and deep Actor-Critic neural networks with a memory module, respectively, to achieve autonomous exploration in simple environments. However, the networks struggled to achieve effective convergence due to their end-to-end exploration approach. Therefore, Li et al. [11] proposed a generic exploration framework containing decision, planning, and mapping modules by decomposing the exploration process and using an edge segmentation task in the decision module to speed up the convergence of the neural network. Nevertheless, the action points of this algorithm were obtained by rasterized sampling on the occupancy map and thus possessed a large action space, which undoubtedly increased the computational cost of the system. Zhu et al. [12] used DRL to predict the order of visits to unexplored subregions and used the NBV selection algorithm to find optimal target points, yet this approach can only be utilized in regular structures such as office buildings. Niroui et al. [28] combined DRL with a frontier-based method by using the Asynchronous Advantage Actor-Critic (A3C) algorithm and frontier point locations for training, enhancing the robot's adaptability to unknown environments. However, this algorithm is not stable enough and often suffers from inefficient exploration in the latter stages of exploration.

Among all the effective works, we note that few researchers consider the regional legacy issues during exploration. A common way is to stop exploration in time when the exploration ratio reaches a certain threshold to avoid inefficiencies later in the exploration process. For example, in recent work, Zagradjanin et al. [8] and Li et al. [11] used the exploration ratio to define a hyperparameter for balancing the path length and exploration efficiency during exploration. As far as we know, we are the first team trying to address the regional legacy issues in the exploration process.

3. System Overview and Problem Formulation

3.1. System Overview

Figure 1 shows the complete system pipeline. The occupancy map with the robot position is used as the input to the system, and the candidate action points are determined by BO sampling [47,48]. The candidate action points are calculated with the occupancy map, robot position, and candidate action points according to the local exploration strategy and the global perception strategy. The local exploration strategy considers the local occupancy map and the candidate action points as its input, and the global perception strategy chooses the global occupancy map and the candidate action points as its input. The evaluated candidate action points are weighted together, and the highest-scoring candidate action point is selected as the target point for the next movement. The current position of the robot and the target point are sent to the navigation module, which plans the optimal path and drives the robot to the target point for exploration.

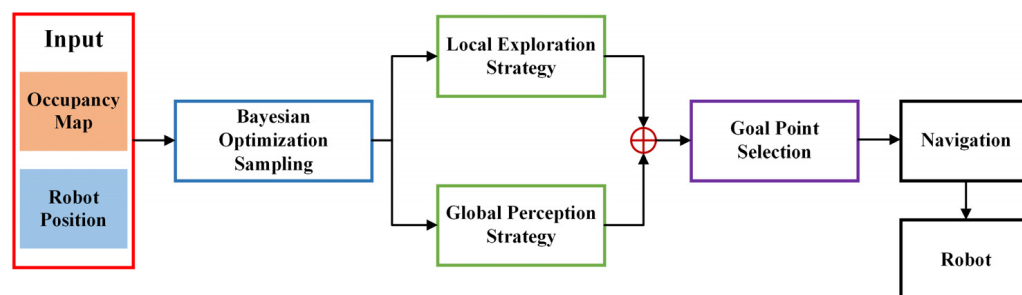


Figure 1. Proposed system pipeline diagram.

3.2. Problem Formulation

The goal of autonomous exploration by a robot can be defined as finding an optimal exploration path L^* for the construction of the whole unknown map [12]. We decompose the optimal exploration path L^* into a set of target points $F = \{X_1, X_2, \dots, X_T\}$ so that the goal of autonomous exploration can be converted into finding a set of optimal points F^* . The robot can complete the construction of the whole unknown map by traversing the target points in the point set in turn. The best points set F^* can be expressed as follows:

$$F^* = \underset{X_{1:T}}{\operatorname{argmin}} \left\{ H(m_t) + \sum_{t=1}^T L(X_t|m_t) \right\} \quad (1)$$

where $H(m_t)$ is the Shannon entropy of the currently occupied map m_t , T is the final time step, and L is the length between X_t and X_{t-1} on the path.

In fact, the optimal points finding process in autonomous exploration can be seen as a sequential decision process, where the robot chooses a series of actions to maximize the accumulation of future rewards [14]. Markov decision process (MDP) [20,26] is a commonly used decision framework used in solving the sequential decision problems mentioned above and can be formulated as a tuple $\langle S, A, T, G, \gamma \rangle$. S is a finite set of states, and in the proposed system, $s \in S$ is the information currently known by the robot, including the current position, current observations of the environment, and candidate action points on the current environment. A is the set of actions that the agent can perform. $T(s'|s, a)$ gives the distribution of transformations over the current state s and the set of actions a to the next state s' . G is the future discounted reward, which is denoted as follows:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots + \gamma^{T-t} r_T = \sum_{t'=t}^T \gamma^{t'-t} r_{t'} \quad (2)$$

where G_t refers to the expected reward between the time t and the final time step T , and γ is the discount factor.

4. Bayesian Optimization Based Sampling

This section describes how the process of determining a robot's candidate action points through BO sampling is performed. The overall process is shown in Figure 2. Firstly, N_{GPR} sampling points from a 2D Sobel sequence [49] are selected to form the initial training sample. GPR [50,51] predicts the posterior mean and variance of the currently occupied map based on the initial training sample. The posterior mean represents the predicted MI reward surface of the current occupancy map, and the variance represents the degree of uncertainty of the current predicted MI reward surface. After GPR convergence or the maximum number of iterations N_{BO_max} is reached, we use the posterior mean as the MI reward surface of the currently occupied map and select N_{G_action} sampled points with the highest MI gain as the candidate action points for the robot. To prevent the robot from having no candidate action points in the local environment, we also sample the local map N_{L_BO} times (BO sampling) and select N_{L_action} sample points with the highest MI gain, which are added to the candidate action points of the robot. After several experiments, we set N_{GPR} to 40, N_{BO_max} to 60, N_{G_action} to 20, N_{L_BO} to 20 and N_{L_action} to 5.

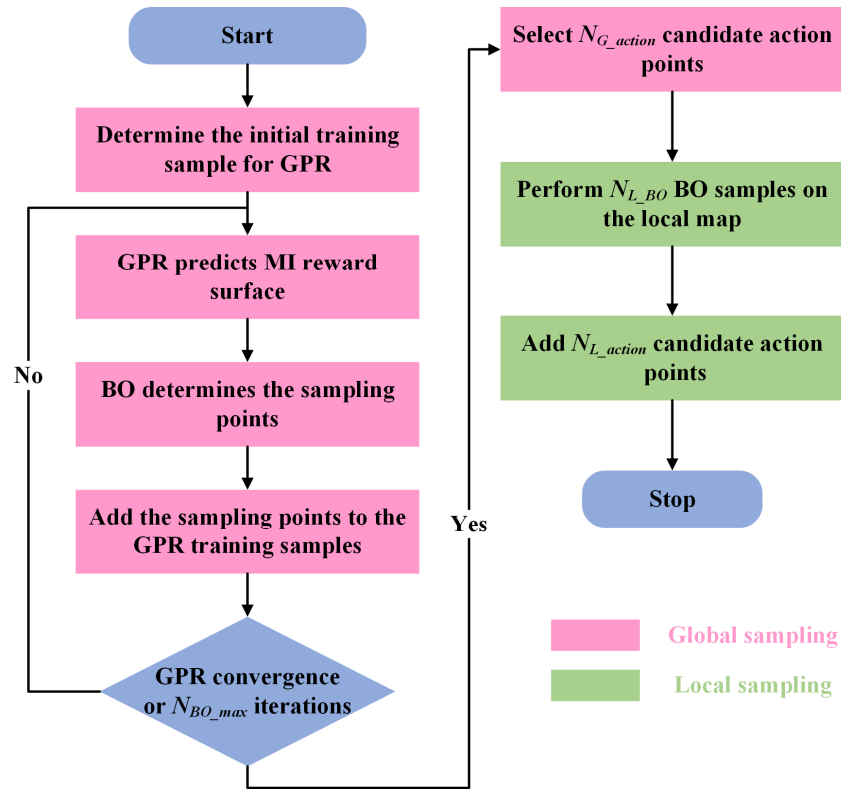


Figure 2. BO sampling flow diagram.

4.1. Mutual Information Gain

In the autonomous exploration process, the robot needs to consider a combination of map and positional uncertainties to select a preferable candidate action point. Since the environment is unknown (because we cannot directly know the uncertainty of the currently built map), we use the Shannon entropy [52,53] as the uncertainty of the currently built map instead. The Shannon entropy is defined on the occupancy grid map m as follows:

$$H(m) = - \sum_i \sum_j p(m_{i,j}) \log p(m_{i,j}) \quad (3)$$

where $p(m_{i,j})$ represents the occupancy probability of the cross grid of row i and column j . Each grid has three occupancy probabilities: idle, occupied, and unknown. Idle and unknown cells do not contribute entropy, and the probability of unknown cells is defined as $p(m_{i,j}) = 0.5$. Each unknown cell contributes one unit of entropy.

MI represents the extent to which the current measurement in the occupancy map reduces entropy, i.e., the reduced uncertainty in the robot position associated with all cells [38]. The MI reward function is defined as follows:

$$I(m_t, x_{t+1}) = H(m_t) - H(m_t | x_{t+1}) \quad (4)$$

where $H(m_t)$ is the entropy of the currently occupied map m_t and $H(m_t | x_{t+1})$ denotes the expected entropy of a new map constructed based on sensor observations after selecting action x_{t+1} on the occupied map m_t . Directly taking action with the maximum MI gain can not guarantee an optimal exploration path [27]. Therefore, we choose several action points with high MI gain as candidate action points and submit them to DRL to select the optimal candidate action points to perform to guarantee the optimal exploration path.

4.2. Gaussian Process Regression

Since the direct computation of MI reward surfaces is costly, we use GPR to predict MI reward surfaces as a way to reduce the computational cost of the system. We assume a set of training data $x = \{x_1, x_2, \dots, x_n\}$, where the corresponding mutual information $I(m, x_i)$ for all $x_i \in x$ has been computed and forms the output set $y = \{y_1, y_2, \dots, y_n\}$. The GPR is constructed using the training data x and its corresponding output y , which is used to predict the MI gain at the specified location of the currently occupied map. The GPR will estimate the output value y_* and the corresponding covariance $cov(y_*)$ associated with the test data x_* by the following equation:

$$y_* = k(x_*, x)[k(x, x) + \sigma_n^2 I]^{-1}y \quad (5)$$

$$cov(y_*) = k(x_*, x_*) - k(x_*, x)[k(x, x) + \sigma_n^2 I]^{-1}k(x, x_*) \quad (6)$$

In the above equation, y_* is the valuation $I(m, x_*)$ of the test data x_* , $cov(y_*)$ is the covariance associated with the output y_* , σ_n^2 is the vector of Gaussian noise variances associated with the training output y , and $k(x, x')$ is the kernel function (also known as the covariance matrix). The commonly used Matérn kernel is utilized to describe the correlation between the input data x and x' [50], which is defined as follows:

$$k(x, x') = \frac{2^{1-v}}{\Gamma(v)} \left(\frac{\sqrt{2v}|x - x'|}{\ell} \right)^v K_v \left(\frac{\sqrt{2v}|x - x'|}{\ell} \right) \quad (7)$$

where v is a parameter used to vary the covariance smoothness, ℓ is the feature length, Γ is the gamma function, and K_v is the modified Bessel function. Compared to another commonly used radial basis function (RBF) kernel, the Matérn kernel can successfully simulate sharp changes in the mutual information reward surface due to the presence of an obstacle.

4.3. Bayesian Optimal Sampling

The candidate sampling actions suggested by BO are calculated using the acquisition function [47]. There are two categories of optimization for the acquisition function: exploration, where selection is made in unexplored regions with high uncertainty, and exploitation, where selection is made in the vicinity of the existing maximum. Any collection function used must be traded off between exploration and exploitation. We use the Gaussian Process Upper Confidence Bound (GP-UCB) algorithm [50] as our collection function, the GP-UCB is expressed as follows:

$$x_{sample}^* = \underset{x \in \mathcal{C}_{action}}{argmax} \mu(x) + \kappa \sigma(x) \quad (8)$$

where κ is the trade-off parameter between exploration and exploitation, $\mathcal{C}_{action}$ is the action point with the exact resolution as the occupied map. $\mu(x)$ and $\sigma(x)$ are the mean and variance predicted by the GPR, and x_{sample}^* denotes the selected optimal sampling point. In the proposed system, we prefer distributing the robot candidate action points evenly across multiple high-value regions rather than concentrating on a single high-value region. Therefore, we configure the acquisition function to favor exploration over exploitation as much as possible.

Figure 3 illustrates how to determine candidate action points for the robot using BO sampling. Figure 3a shows the currently built occupancy map, robot position, and local map. Figure 3b shows the “ground truth” of MI, which is the actual MI reward surface for the current occupancy map. Figure 3c shows the initial training sample for GPR and the predicted MI reward surface based on this. Figure 3d shows the MI reward surface and training samples after 60 BO iterations. Figure 3e shows the MI reward surface and the

local sampling points predicted based on the local map. Figure 3f shows the selected local and global candidate action points.

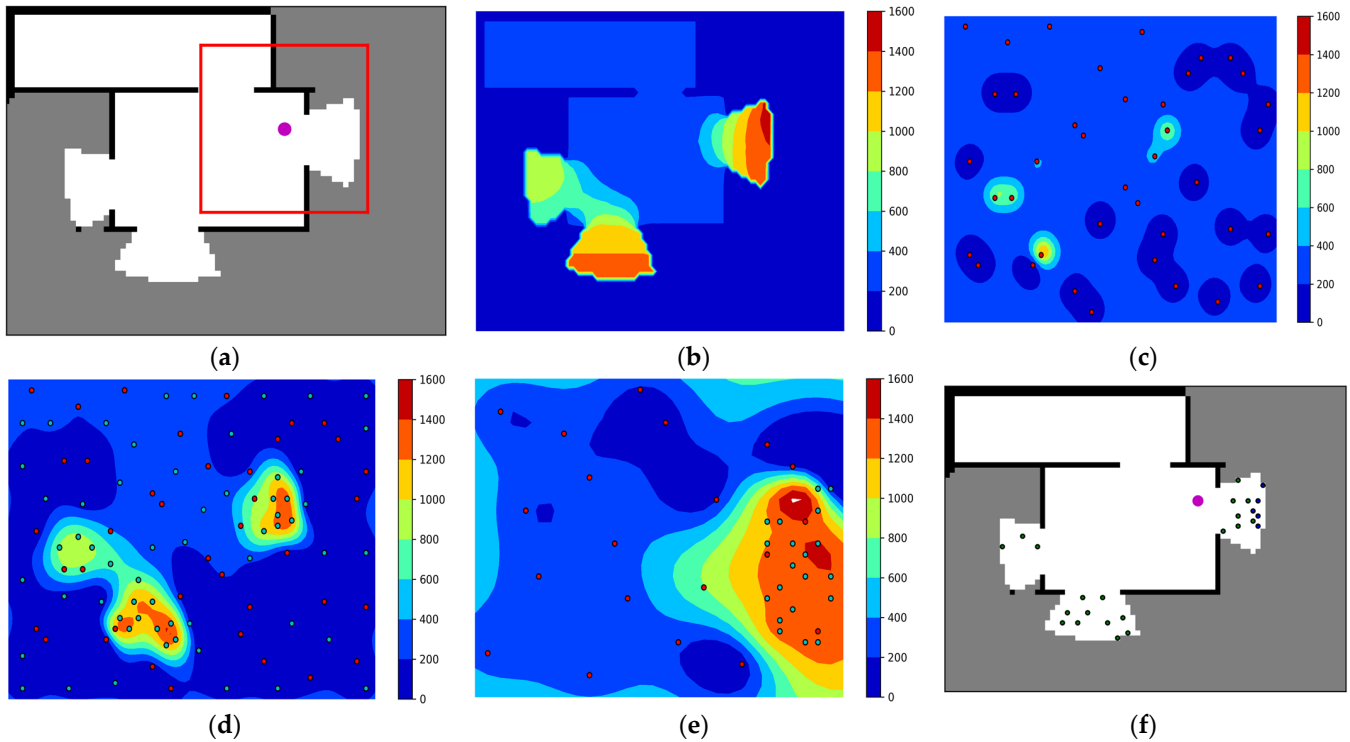


Figure 3. Bayesian optimal sampling process. (a) Current occupancy map, robot position, and local map; (b) Ground Truth: Real MI reward surface; (c) MI reward surface and initial training samples; (d) MI reward surface and training samples after 60 BO iterations; (e) Local MI reward surfaces and local sampling points; (f) Selected candidate action points (global: red, local: blue).

5. DRL-Based Decision Method

In this paper, we use a direct strategy to calculate the unexplored regions with the highest gain in local exploration. Meanwhile, global perception trained using DRL guides local exploration by extracting global information to plan the optimal exploration path.

5.1. Local Exploration and Global Perception

5.1.1. Local Exploration

The objective of local exploration is to find the candidate action point x_{local}^* with the highest MI gain in the current local region to explore the unknown region as fast as possible, and its objective function is expressed as follows:

$$x_{local}^* = \underset{x_t \in L_{action}}{\operatorname{argmax}} I(m_{local}, x_t) \quad (9)$$

where m_{local} represents the current local occupancy map, L_{action} represents the set of candidate action points on the current local map, and x_{local}^* represents the selected local optimal candidate action points. We define a heuristic reward score for the local exploration strategy and normalize the score: $score \in [-1, 1]$. After empirical testing, we set 70% of the maximum possible MI gain as the upper bound of the score $I_{ht}(m_{local}, x)$, as well as normalized the MI gain obtained by the robot after selecting a candidate action point. When the robot is too close to an obstacle after selecting a candidate action point or the MI gain obtained is less than the threshold $I_{lt}(m_{local}, x)$ (this threshold is slightly greater than zero),

it will be given a penalty of -1 . This provides adequate safety for the robot and prevents it from repeatedly exploring the explored regions. The reward score can be expressed as:

$$score_{local} = \begin{cases} 1 & I(m_{local}, x_t) \geq I_{ht}(m_{local}, x) \\ \frac{I(m_{local}, x_t)}{I_{ht}(m_{local}, x)} & I_{lt}(m_{local}, x) < I(m_{local}, x_t) < I_{ht}(m_{local}, x) \\ -1 & I(m_{local}, x_t) \leq I_{lt}(m_{local}, x) \text{ or close to the obstacle} \end{cases}, \quad (10)$$

where $score_{local}$ is the calculated score value from the local candidate action points and $I(m_{local}, x_t)$ denotes the MI gain obtained by selecting the candidate action point x_t on the current local occupancy map m_{local} .

5.1.2. Global Perception

Global perception focuses on small unexplored regions that are closer to the robot. When there are two unexplored regions with similar distances and different gains, global perception prefers to select the unexplored region with the smaller gain to compensate for the regional legacy issues caused by excessive greed in local exploration. Its objective function can be expressed as follows:

$$x_{global}^* = \underset{x_t \in G_{action}}{argmax} \left\{ -\phi \frac{I(m_{global}, x_t)}{I_{ht}(m_{global}, x)} - \varphi \frac{L(x_t)}{L_{max}} \right\} \quad (11)$$

where G_{action} denotes the set of candidate action points on the current global occupancy map, ϕ and φ are weight coefficients, and $\phi < \varphi$, x_{global}^* denotes the selected global optimal candidate action point. $L(\cdot)$ is the A* path length between the robot and this candidate action point. Global perception also has an important function. When local exploration falls into a local optimal solution, global perception will guide the robot to the unexplored region to continue exploration. In addition, global perception is equipped with a terminal action that will decide whether to take a terminal action to stop exploration when the exploration area is determined to be larger than a threshold, the definition of a terminal action and the reward received for taking that action are shown as follows:

$$r_t = \begin{cases} 1 & \text{if the ratio of explored region in map } \rho > 0.95 \\ -0.2 & \text{otherwise} \end{cases} \quad (12)$$

In the above equation, ρ is a hyperparameter that indicates the proportion of the region to be explored. During the empirical evaluation, we set it to 0.95.

5.2. Network Structure

A combined image is constructed for the global perception network inputs, including the current occupancy map, robot position, and the candidate action points on the current occupancy map. The robot position and candidate action points are each plotted on a blank map of the same size as the occupancy map. The aim is to enable the network to take full advantage of all the critical information useful for decision-making.

The network structure is shown in Figure 4. Since the superior performance of feature extracting, we use four convolutional layers to extract features from the input image. After each convolutional operation, we use a rectified linear unit (RELU) to remove the negative feature values. The features extracted by the convolutional layers are classified by a fully connected layer and transferred to a long short-term memory (LSTM) unit to ensure that the network considers previous critical information when making decisions. The output of the LSTM unit is used directly by the Actor and Critic layers. The output of the Actor layer is passed through the Sigmoid activation function to obtain a weight parameter $\omega \in (0, 1)$. The Filter layer uses the weight parameter ω in the evaluation function to evaluate the score value of each input candidate action point. The evaluation function consists of the

A* path length between the robot and the candidate action point and the MI gain of the candidate action point [28]. The evaluation function is formulated as follows:

$$score_{global} = \omega \times \bar{g} + (1 - \omega)(1 - \bar{d}) \quad (13)$$

where $score_{local}$ is the calculated score value from the global candidate action points, $\bar{d}$ and $\bar{g}$ are the normalized distance and MI gain. The score values of candidate action points evaluated by both the local exploration strategy and the global perception strategy are weighted and superimposed by the score values of both. The superimposition equation is shown below:

$$score_{finsh} = score_{local} + \eta score_{global} \quad (14)$$

where $score_{local}$ and $score_{global}$ denote the scores evaluated by the local exploration strategy and the global perception strategy, respectively, and η is the scale factor. Once the scores of all candidate action points are determined, the robot will select the candidate action point with the highest score for the next exploration.

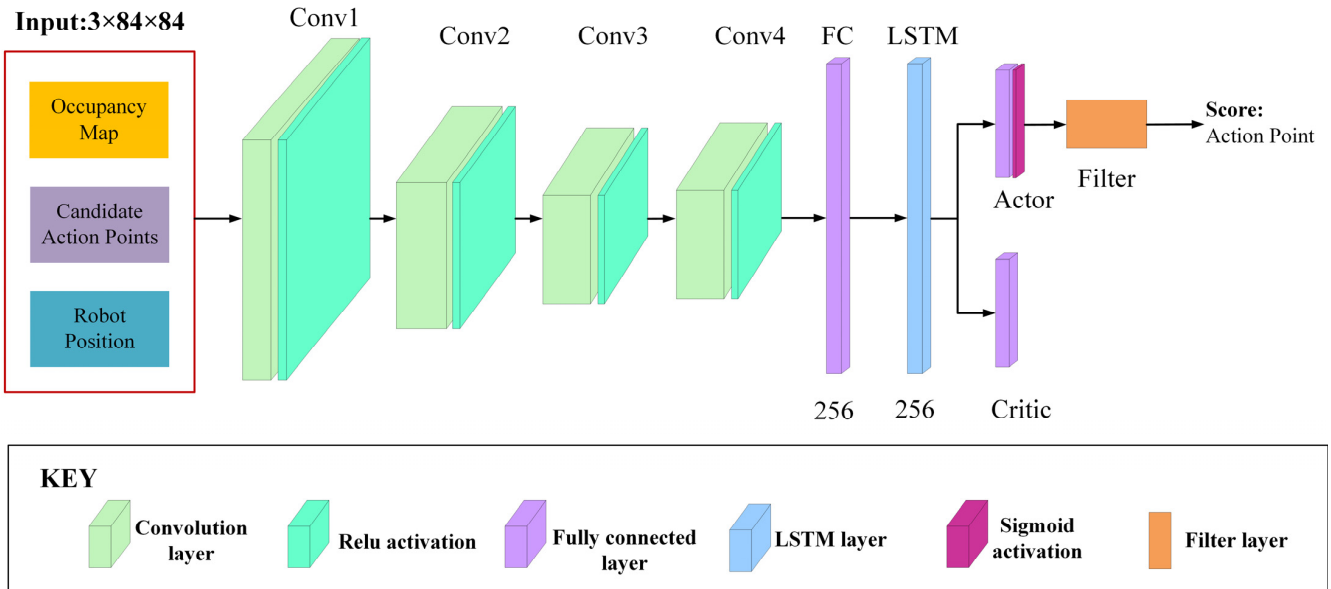


Figure 4. System network structure.

5.3. Asynchronous Advantage Actor–Critic Algorithm

We train our network with the A3C algorithm [54,55] based on the Actor–Critic framework. It combines value-based and policy-based methods and has two outputs, one corresponding to the policy π and another output of the state value V . The loss function of the A3C algorithm typically consists of three terms [56]: the policy gradient loss, value residual, and strategy entropy regularization, denoted respectively as follows:

$$Policy_{loss} = - \mathbb{E}_{\tau \sim \pi_{\theta}} \sum_{t=0}^T \log \pi_{\theta}(a_t | s_t) A(s_t, a_t; \theta, \theta_v) \quad (15)$$

$$Value_{loss} = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\frac{1}{2} \sum_{t=0}^T (R_t - V(s_t; \theta_v))^2 \right] \quad (16)$$

$$Entropy_{loss} = -H(\pi_{\theta'}(a_t|s_t)) \quad (17)$$

where the strategy entropy regularization term ensures the diversity of actions and enhances the robot's ability to explore the environment. The total loss function is shown as follows:

$$Total_{loss} = Policy_{loss} + \alpha * Value_{loss} + \beta * Entropy_{loss} \quad (18)$$

In the above equations, H is the entropy of the strategy, α is the weight coefficient of the value residual term, β is the weight coefficient of the strategy entropy regularization term, and θ and θ_v are the parameters of the strategy and value functions, respectively. $A(s_t, a_t; \theta, \theta_v)$ is an estimate of the dominance function, indicating the dominance of the chosen action a_t relative to the average in state s_t , which can be expressed as follows:

$$A(s_t, a_t; \theta, \theta_v) = R_t - V(s_t; \theta_v) \quad (19)$$

$$R_t = G_t + \gamma^k V(s_{t+k}; \theta_v) \quad (20)$$

where G_t and γ have the same meaning as in Equation (3), k is the total steps from the current step t to the final step T , and V is an estimated function of the additional benefit gained by taking action in a certain state.

The A3C algorithm can start multiple threads (workers), each of which synchronizes the latest network parameters from A3C's global network and performs independent exploration, training, and learning [56]. When this exploration ends or reaches the maximum step size T , the gradient accumulated by itself is updated to A3C's global network. Then, the latest network parameters are synchronized and repeated until convergence. θ and θ_v are updated as follows:

$$d\theta = d\theta + \nabla_{\theta'} \log \pi(a_t|s_t; \theta') (R_t - V(s_t; \theta_v)) + \beta \nabla_{\theta'} H(\pi(a_t|s_t; \theta')), \quad (21)$$

$$\theta_v = d\theta_v + \partial (R_t - V(s_t; \theta_v))^2 / \partial \theta_v, \quad (22)$$

where H is the entropy of the strategy and β is the weighting factor of the strategy entropy regularization term.

6. Experiments

In this section, we train our algorithm in a simulation environment, then compare it with different algorithms to evaluate our algorithm's performance and analyze its generalizability on maps of varying layouts and sizes. Finally, we summarize the experimental results.

6.1. Evaluation Indicators

To analyze the performance of our algorithm, we use the explored region rate, average path length, and exploration efficiency as evaluation metrics, which are expressed as follows:

1. Explored region rate ζ : This metric evaluates the completeness of the map built by the robot during exploration, and it is defined as

$$\zeta = \frac{N_{f_explored}}{N_{f_real}} \quad (23)$$

where $N_{f_explored}$ is the number of free spaces explored and N_{f_real} is the number of free spaces in the actual map;

2. Average path length $\bar{L}$: This metric evaluates the average path length taken by the robot in the total set of trials, and it is defined as

$$\bar{L} = \frac{\sum L_i}{N_{episodes}} \quad (24)$$

where $N_{episodes}$ is the number of trials, and L_i refers to the length of the path moved on the i th trial;

3. Exploration efficiency K : This metric evaluates the entropy reduction of the robot after moving a unit distance on average over the total set of trials, and it is defined as

$$K = \frac{\sum (H(m_{T_i}) - H(m_0))}{\sum L_i} \quad (25)$$

where $H(m)$ is the entropy of the occupied map m , and T_i and L_i refer to the number of steps moved and the length of the path in the i th trial, respectively.

6.2. Simulation Setup

We design a Python-based simulation platform to train the intelligent robot, using a beam-based sensor model to simulate the robot's exploration of its environment and an A* algorithm to plan the robot's path to a target point. We assume that the robot can move omnidirectionally and has an all-round 360° field of view with a resolution of 0.5° and that the sensor can detect a circular area with a radius of 20 pixels. We train and test our algorithm on maps of different layouts and sizes selected from HouseExpo (a sizeable indoor layout dataset) constructed in [57]. Figure 5 and Table 1 show the details of the maps.

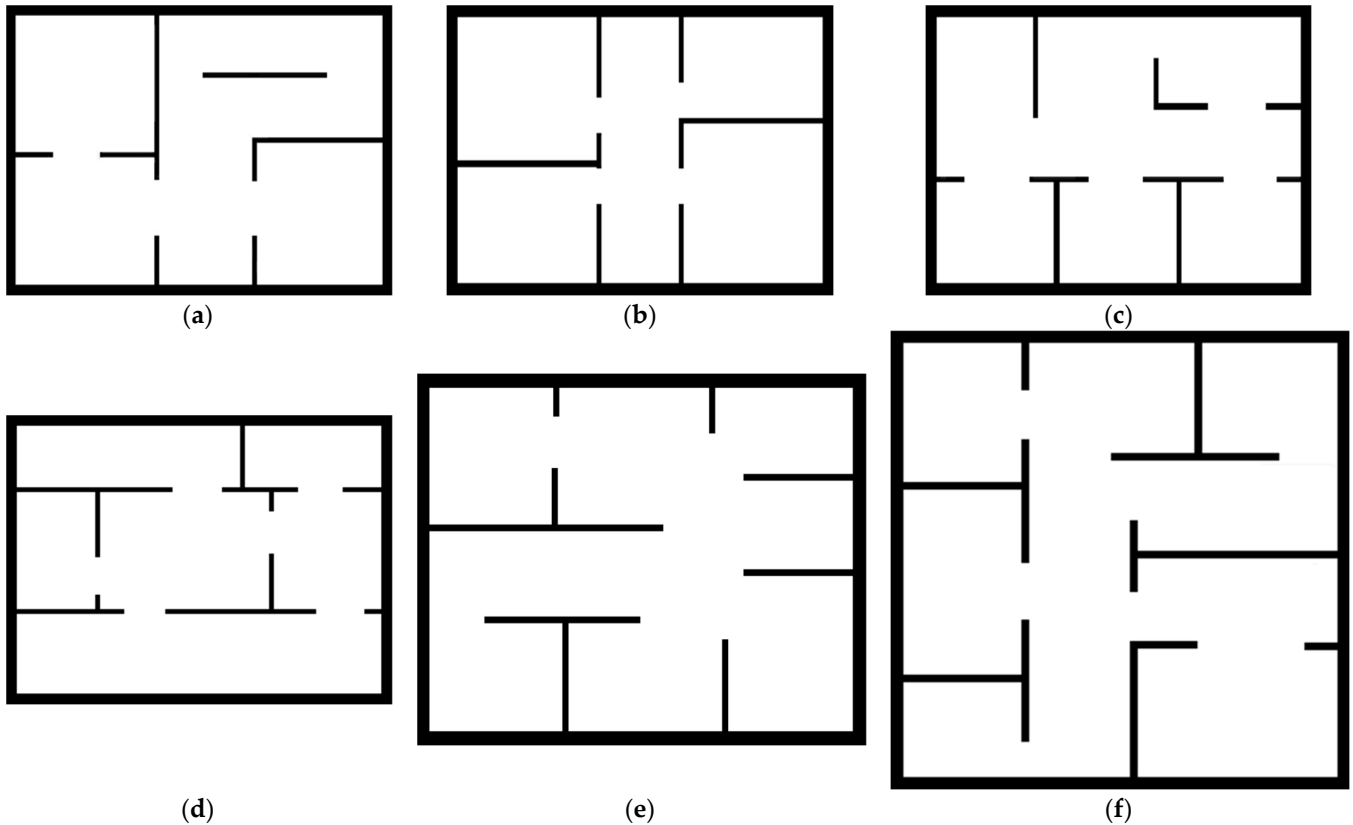


Figure 5. Training and test maps. (a) Train map 1; (b) Train map 2; (c) Train map 3; (d) Test map 1; (e) Test map 2; (f) Test map 3.

Table 1. Details of training and test maps.

Map Name	Resolution (Pixel)
Train map 1	60×80
Train map 2	60×80
Train map 3	60×80
Test map 1	60×80
Test map 2	77×93
Test map 3	95×95

6.3. Training Setup

We train our algorithms on an Intel (R) Core (TM) i9-9900K CPU at 3.60 GHz without using a GPU for acceleration. We train three agents simultaneously to interact with the environment in training maps 1, 2, and 3, respectively, with the robot's initial position being randomized for each exploration. After several experiments, the hyperparameters of the A3C algorithm and network structure used for tests are shown in Tables 2 and 3, respectively.

Table 2. A3C network hyperparameters.

Hyperparameters	Value
Number of parallel environments	3
Number of minibatches T	30
Number of episodes	100,000
Learning rate	0.0001
Learning rate decay policy	Polynomial decay
Optimization algorithm	Adam
Value loss coefficient α	0.5
Entropy coefficient β	0.01
Discount factor γ	0.99
Exploration region rate ρ	0.95
Score proportion factor η	0.75
Covariance smoothness coefficient v	2.5
BO tradeoff coefficient κ	5.0

Table 3. Network structure hyperparameters.

Layer	Hyperparameters
Conv1	Output = 16, Kernel = 8, Stride = 4, Padding = Valid
Conv2	Output = 32, Kernel = 4, Stride = 2, Padding = Valid
Conv3	Output = 32, Kernel = 3, Stride = 2, Padding = Same
Conv4	Output = 32, Kernel = 3, Stride = 2, Padding = Same
FC	Output size = 256
LSTM	Output size = 256

6.4. Algorithm Comparison

We use several heuristic-based and learning-based methods as baselines to evaluate the performance of our method, formulated as below:

1. NF (nearest frontier). The method is explored by selecting the nearest frontier point to the robot as the target point.
2. MI (maximum mutual information). This method calculates the MI gain for each action point and explores it by selecting the action point with the maximum MI gain.

3. LS (local exploration strategy). We modified the source code provided by Chen et al. [13] and applied it to our environment. The action points of the method consist of 40 Sobel sampling points in the vicinity of the robot, and the goal is to find and execute the action point with the highest gain in the local area. When the LS reaches a “dead end”, it is guided to the nearest frontier point using the FR strategy.
4. GS (global exploration strategy). We replicate the method proposed by Niroui et al. [28] in our setting. The method uses the frontier points of the currently occupied map as action points, with the goal of maximizing the total information gained from the robot’s exploration path.

In these methods, NF and MI are heuristic-based methods, and LS and GS are learning-based methods. Heuristic-based methods explore unknown environments based on global optimization functions, and learning-based methods explore unknown environments based on autonomously learned strategies. We first tested on Test Map 1, shown in Figure 5, which was the same size as the training map but had a different layout. All methods were tested 30 times with a random initial location, and each exploration does not stop until the environment is fully explored. The combined performance is shown in Figure 6. The curves represent the proportion of explorations at the average path length.

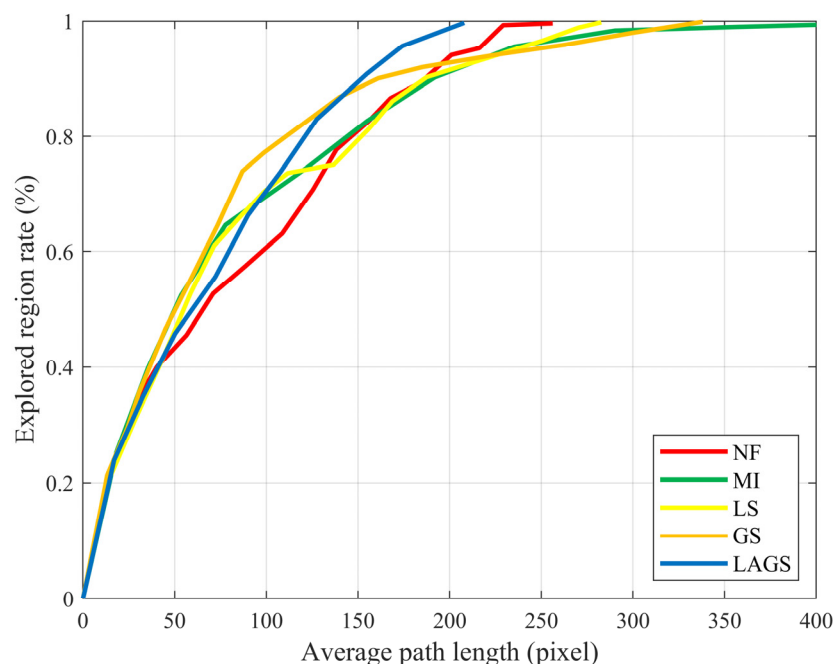


Figure 6. Proportion of exploration at average path length.

As can be seen in Figure 6, the LAGS algorithm had better performance and shorter paths than other algorithms. The average path length of LAGS was 18.8%, 49.3%, 26.4%, and 38.4% shorter than the average path length of NF, MI, LS, and GS, respectively. We can see that MI, LS, and GS showed good exploration efficiency in the early exploration stage. However, the exploration efficiency decreased to varying degrees in the latter stage of exploration due to the influence of the regional legacy issues, and the decrease was especially obvious for MI and GS. LAGS and NF were less affected by the regional legacy issues. They maintained a more stable exploration efficiency throughout the exploration process, with the overall exploration efficiency of LAGS being significantly higher than that of NF.

In order to better analyze the exploration performance of LAGS and other algorithms, we compared the exploration paths of these algorithms at the initial positions (10,10), (70,35), and (35,50), respectively. The comparison results are shown in Figure 7. Compared with other algorithms, LAGS could find more reasonable exploration paths at different

initial positions, and there were almost no duplicate explorations and redundant paths. Due to MI's one-sided exploration strategy for high gain and LS's lack of global information, there was always redundancy in their exploration paths. NF depends on the map's features due to its non-learning exploration strategy, which makes it exhibit different exploration performances at different initial locations. However, GS could plan a reasonable exploration path in some cases, but it uses more path cost to complete the final exploration in the later stages of exploration.

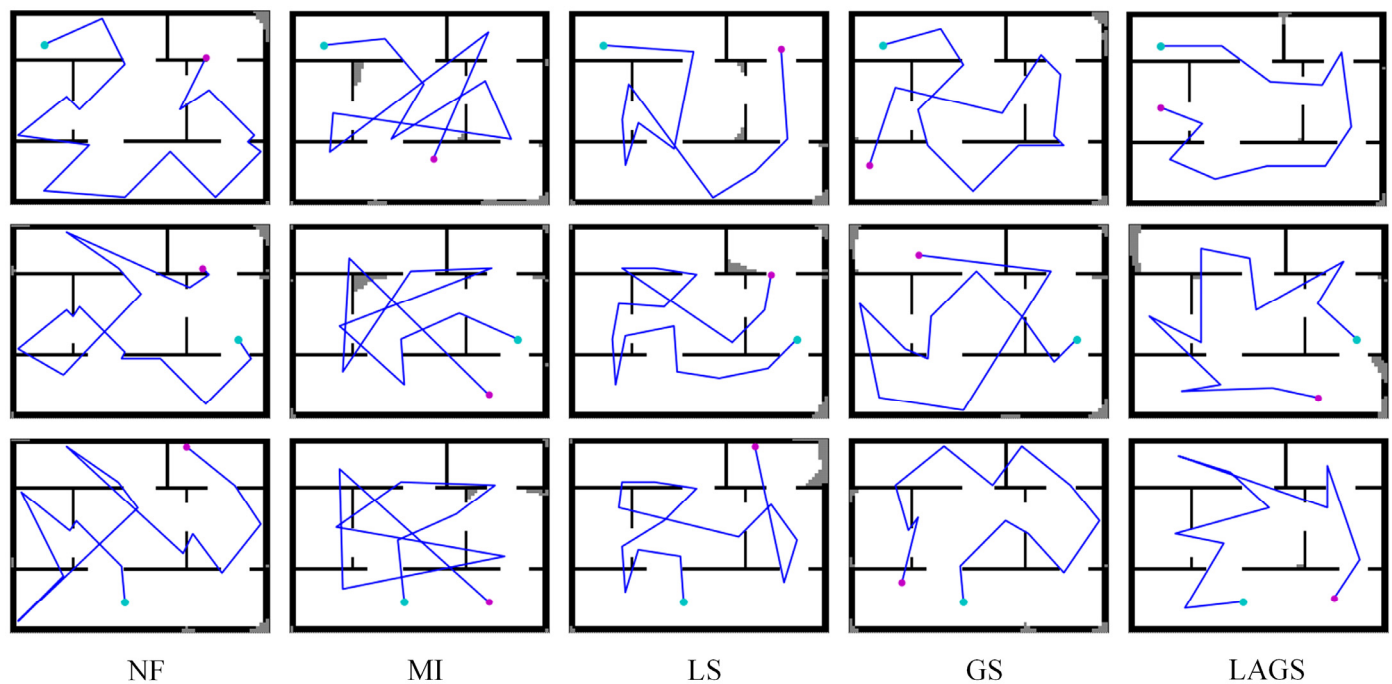


Figure 7. Exploration paths for each algorithm at initial positions (10,10) (top), (70,35) (middle), and (35,50) (bottom), respectively. The cyan point is the initial position of the exploration, the magenta point is the end position of the exploration, and the blue line is the line between two adjacent exploration target points and is not the actual path of movement of the robot.

LAGS can plan reasonable exploration paths, mainly because it considers and solves the regional legacy issues in the exploration process. We visualize the exploration process of the LAGS algorithm at an initial position of (10,10), which is depicted in Figure 8. Early in the exploration, the robot selects target points with high MI gain for exploration to explore the environment faster. In steps four and nine, the robot has several selectable unexplored regions. Global perception guides the robot to select unexplored regions that are close together and smaller in size to avoid regional legacy issues later in exploration. After step 12, the LAGS takes a terminal action to stop exploration.

6.5. Summary Analysis

To analyze the generalizability of LAGS, we tested it in test maps 1, 2, and 3 with different layouts and sizes, as shown in Figure 5. We tested each test map 50 times with a random initial position. The algorithm chooses at its discretion whether to take a terminal action to stop exploration during the exploration process. The test results are shown in Table 4 and Figure 9.

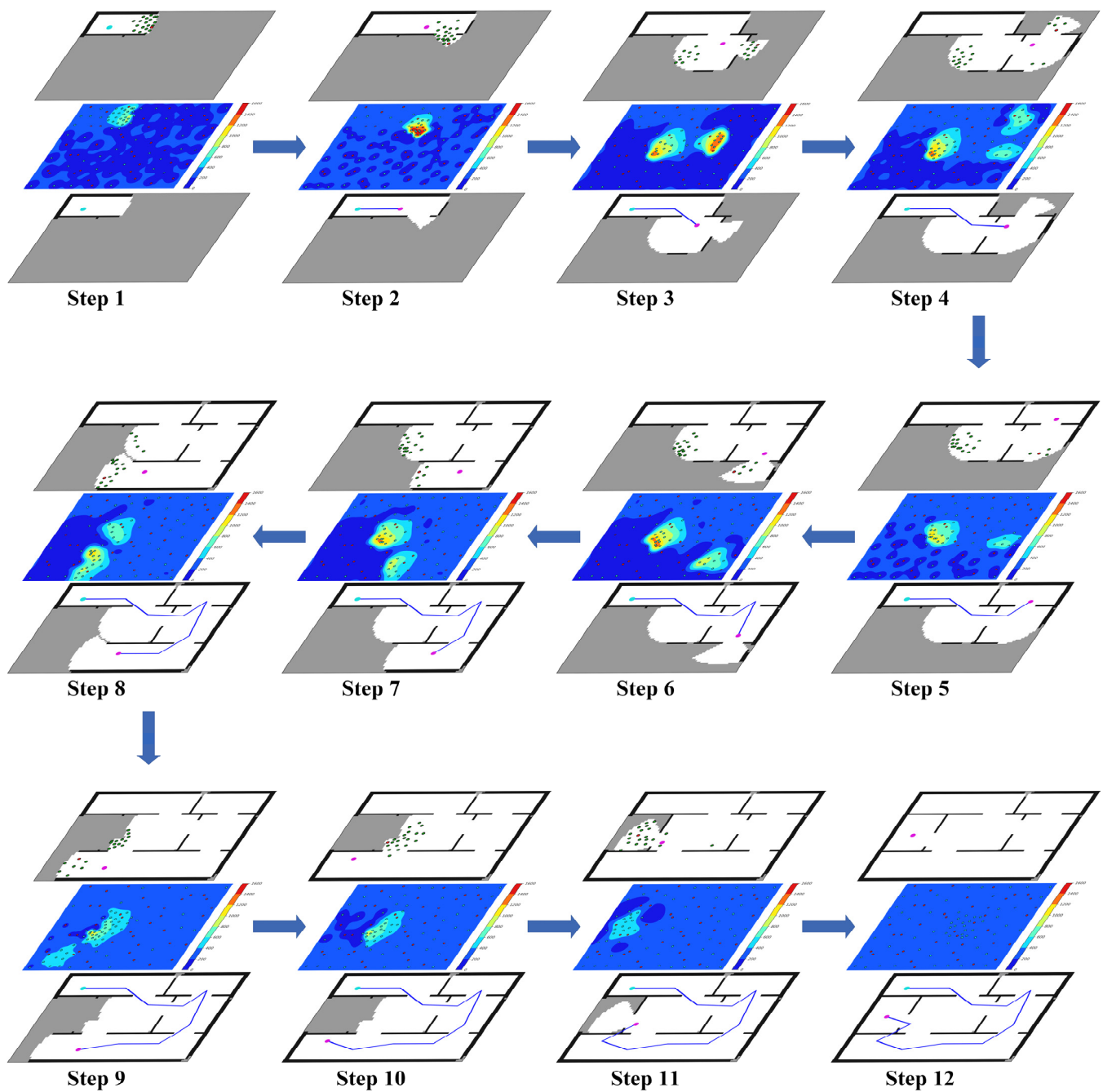
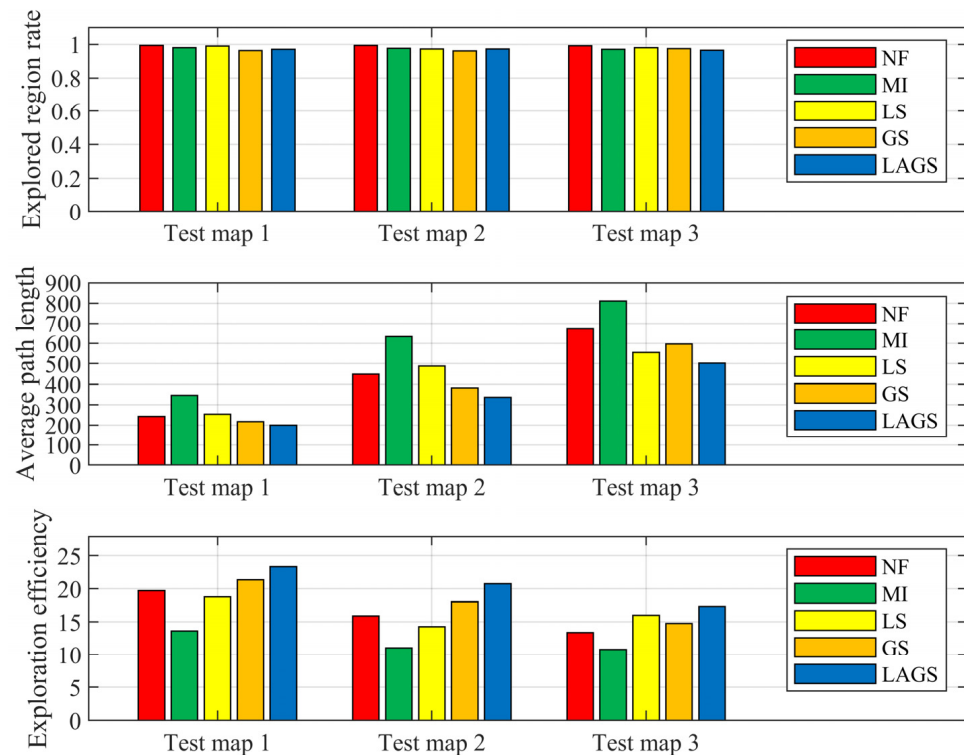


Figure 8. The exploration process of the LAGS algorithm at an initial position of (10,10). The entire process consists of 12 steps, each with three layers; the lower layer includes the current occupancy map, the robot's initial position (cyan), the robot's current position (magenta), and the line between previously adjacent target points (blue), the middle layer includes the MI reward surface, and sampling points predicted based on the current occupancy map; and the upper layer includes the robot's current position (magenta), multiple candidate action points (green), and candidate target points (red).

Table 4. Comparison of statistical results between different algorithms.

	Average Path Length			Explored Region Rate			Exploration Efficiency		
	Test Map 1	Test Map 2	Test Map 3	Test Map 1	Test Map 2	Test Map 3	Test Map 1	Test Map 2	Test Map 3
NF	242.19	448.4	670.79	0.994	0.995	0.993	19.7	15.89	13.36
MI	344.86	633.07	809.82	0.98	0.976	0.97	13.64	11.04	10.81
LS	253.69	488.45	554.03	0.991	0.972	0.981	18.75	14.25	15.98
GS	216.61	381.47	595.76	0.963	0.961	0.975	21.34	18.04	14.77
LAGS	199.57	335.79	501.96	0.97	0.973	0.965	23.33	20.75	17.35

**Figure 9.** Statistical results of the different algorithms in the test maps.

As can be seen from Table 4 and Figure 9, LAGS has the shortest exploration path and highest exploration efficiency in all the tested maps. NF and MI have higher exploration ratios because the heuristic methods keep exploring the environment until there are no more available points. MI has the longest exploration path and the lowest exploration efficiency due to its one-sided exploration strategy of pursuing high-gain regions. LS also exhibits a high proportion of exploration due to the inclusion of the FR strategy in its exploration strategy, which continuously directs it to the nearest frontier point for exploration. The GS has significant performance degradation in the later stages of exploration, but has good exploration efficiency in the smaller-sized test maps 1 and 2. However, in the larger test map 3, exploration efficiency dropped significantly and is not as good as the LS.

The two experiments described above lead us to the following conclusions:

1. LAGS has a stronger exploration performance. In cases with different initial positions, LAGS can solve the regional legacy issues and plan reasonable exploration paths during the exploration process. In addition, LAGS can achieve a higher exploration ratio for the same average path length compared to other algorithms.
2. LAGS has good robustness and environmental adaptability. In test experiments in environments of various sizes and layouts, LAGS achieves better performance with shorter exploration paths and higher exploration efficiencies compared to other algorithms.

7. Conclusions

In this paper, we analyze the impact of the regional legacy issues prevalent in existing exploration algorithms on exploration efficiency and propose a LAGS algorithm to address the problem that a single exploration strategy is challenging to balance between optimal exploration paths and environmental robustness. The algorithm addresses the redundant path and region legacy issues by integrating a local exploration strategy with a global perception strategy to improve exploration efficiency. Our experiments, conducted in environments with various layouts and sizes, demonstrate the robustness, environmental adaptability, and high exploration efficiency of the LAGS algorithm. The algorithm allows the exploration of unknown environments faster, to speed up the search for victims in missions such as personnel search and rescue and disaster reconnaissance. In addition, we use GPR and BO sampling points as candidate action points for the robot, which provides a new method for the robot to select candidate action points.

In future research, we will consider speeding up the training process of the A3C network by introducing empirical information gained during human deployment into the algorithm, e.g., by applying inverse reinforcement learning techniques. Secondly, we aim to optimize the exploration strategy further to reduce the system's decision-making time. Finally, autonomous exploration in unknown environments with dynamic obstacles is also one of our possible subsequent research directions.

Author Contributions: Conceptualization, A.F. and J.X.; methodology, A.F.; validation, Y.X. and Y.S.; writing—original draft preparation, A.F., Y.X. and Y.S.; writing—review and editing, X.W. and B.J. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the National Nature Science Foundation of China under grant No. 61974073 and the Postgraduate Research and Practice Innovation Program of Jiangsu Province under grant No. SJCX22_0259.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The raw/processed data required to reproduce these findings cannot be shared at this time as the data also forms part of an ongoing study.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Krzysia, R.; Butail, S. Information-Based Control of Robots in Search-and-Rescue Missions With Human Prior Knowledge. *IEEE Trans. Hum. Mach. Syst.* **2021**, *52*, 52–63. [\[CrossRef\]](#)
2. Zhai, G.; Zhang, W.; Hu, W.; Ji, Z. Coal mine rescue robots based on binocular vision: A review of the state of the art. *IEEE Access* **2020**, *8*, 130561–130575. [\[CrossRef\]](#)
3. Zhang, J. Localization, Mapping and Navigation for Autonomous Sweeper Robots. In Proceedings of the 2022 International Conference on Machine Learning and Intelligent Systems Engineering (MLISE), Guangzhou, China, 5–7 August 2022; pp. 195–200.
4. Luo, B.; Huang, Y.; Deng, F.; Li, W.; Yan, Y. Complete coverage path planning for intelligent sweeping robot. In Proceedings of the 2021 IEEE Asia-Pacific Conference on Image Processing, Electronics and Computers (IPEC), Dalian, China, 14–16 April 2021; pp. 316–321.
5. Seenu, N.; Manohar, L.; Stephen, N.M.; Ramanathan, K.C.; Ramya, M. Autonomous Cost-Effective Robotic Exploration and Mapping for Disaster Reconnaissance. In Proceedings of the 2022 10th International Conference on Emerging Trends in Engineering and Technology-Signal and Information Processing (ICETET-SIP-22), Nagpur, India, 29–30 April 2022; pp. 1–6.
6. Narayan, S.; Aquif, M.; Kalim, A.R.; Chagarlamudi, D.; Harshith Vignesh, M. Search and Reconnaissance Robot for Disaster Management. In *Machines, Mechanism and Robotics*; Springer: Berlin/Heidelberg, Germany, 2022; pp. 187–201.
7. Perkasa, D.A.; Santoso, J. Improved Frontier Exploration Strategy for Active Mapping with Mobile Robot. In Proceedings of the 2020 7th International Conference on Advance Informatics: Concepts, Theory and Applications (ICAICTA), Tokoname, Japan, 8–9 September 2020; pp. 1–6.
8. Zagradjanin, N.; Pamucar, D.; Jovanovic, K.; Knezevic, N.; Pavkovic, B. Autonomous Exploration Based on Multi-Criteria Decision-Making and Using D* Lite Algorithm. *Intell. Autom. Soft Comput.* **2022**, *32*, 1369–1386. [\[CrossRef\]](#)

9. Liu, J.; Lv, Y.; Yuan, Y.; Chi, W.; Chen, G.; Sun, L. A prior information heuristic based robot exploration method in indoor environment. In Proceedings of the 2021 IEEE International Conference on Real-time Computing and Robotics (RCAR), Xining, China, 15–19 July 2021; pp. 129–134.
10. Zhong, P.; Chen, B.; Lu, S.; Meng, X.; Liang, Y. Information-Driven Fast Marching Autonomous Exploration With Aerial Robots. *IEEE Robot. Autom. Lett.* **2021**, *7*, 810–817. [[CrossRef](#)]
11. Li, H.; Zhang, Q.; Zhao, D. Deep reinforcement learning-based automatic exploration for navigation in unknown environment. *IEEE Trans. Neural Netw. Learn. Syst.* **2019**, *31*, 2064–2076. [[CrossRef](#)]
12. Zhu, D.; Li, T.; Ho, D.; Wang, C.; Meng, M.Q.-H. Deep reinforcement learning supervised autonomous exploration in office environments. In Proceedings of the 2018 IEEE International Conference on Robotics and Automation (ICRA), Brisbane, QLD, Australia, 21–25 May 2018; pp. 7548–7555.
13. Chen, F.; Bai, S.; Shan, T.; Englot, B. Self-learning exploration and mapping for mobile robots via deep reinforcement learning. In Proceedings of the AIAA Scitech 2019 Forum, San Diego, CA, USA, 7–11 January 2019; p. 0396.
14. Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A.A.; Veness, J.; Bellemare, M.G.; Graves, A.; Riedmiller, M.; Fidjeland, A.K.; Ostrovski, G. Human-level control through deep reinforcement learning. *Nature* **2015**, *518*, 529–533. [[CrossRef](#)]
15. Ramezani Dooraki, A.; Lee, D.-J. An end-to-end deep reinforcement learning-based intelligent agent capable of autonomous exploration in unknown environments. *Sensors* **2018**, *18*, 3575. [[CrossRef](#)]
16. Ramakrishnan, S.K.; Al-Halah, Z.; Grauman, K. Occupancy anticipation for efficient exploration and navigation. In Proceedings of the European Conference on Computer Vision, Glasgow, UK, 23–28 August 2020; pp. 400–418.
17. Chaplot, D.S.; Gandhi, D.; Gupta, S.; Gupta, A.; Salakhutdinov, R. Learning to explore using active neural slam. *arXiv* **2020**, arXiv:2004.05155.
18. Hu, J.; Niu, H.; Carrasco, J.; Lennox, B.; Arvin, F. Voronoi-based multi-robot autonomous exploration in unknown environments via deep reinforcement learning. *IEEE Trans. Veh. Technol.* **2020**, *69*, 14413–14423. [[CrossRef](#)]
19. Surmann, H.; Jestel, C.; Marchel, R.; Musberg, F.; Elhadj, H.; Ardani, M. Deep reinforcement learning for real autonomous mobile robot navigation in indoor environments. *arXiv* **2020**, arXiv:2005.13857.
20. Zhang, J.; Tai, L.; Liu, M.; Boedecker, J.; Burgard, W. Neural slam: Learning to explore with external memory. *arXiv* **2017**, arXiv:1706.09520.
21. Peake, A.; McCalmon, J.; Zhang, Y.; Myers, D.; Alqahtani, S.; Pauca, P. Deep Reinforcement Learning for Adaptive Exploration of Unknown Environments. In Proceedings of the 2021 International Conference on Unmanned Aircraft Systems (ICUAS), Athens, Greece, 15–18 June 2021; pp. 265–274.
22. Wu, Z.; Yao, Z.; Lu, M. Deep-Reinforcement-Learning-Based Autonomous Establishment of Local Positioning Systems in Unknown Indoor Environments. *IEEE Internet Things J.* **2022**, *9*, 13626–13637. [[CrossRef](#)]
23. Chen, Z.; Subagdja, B.; Tan, A.-H. End-to-end deep reinforcement learning for multi-agent collaborative exploration. In Proceedings of the 2019 IEEE International Conference on Agents (ICA), Jinan, China, 18–21 October 2019; pp. 99–102.
24. Cimurs, R.; Suh, I.H.; Lee, J.H. Goal-Driven Autonomous Exploration Through Deep Reinforcement Learning. *IEEE Robot. Autom. Lett.* **2021**, *7*, 730–737. [[CrossRef](#)]
25. Lee, W.-C.; Lim, M.C.; Choi, H.-L. Extendable Navigation Network based Reinforcement Learning for Indoor Robot Exploration. In Proceedings of the 2021 IEEE International Conference on Robotics and Automation (ICRA), Xi'an, China, 30 May–5 June 2021; pp. 11508–11514.
26. Song, Y.; Hu, Y.; Zeng, J.; Hu, C.; Qin, L.; Yin, Q. Towards Efficient Exploration in Unknown Spaces: A Novel Hierarchical Approach Based on Intrinsic Rewards. In Proceedings of the 2021 6th International Conference on Automation, Control and Robotics Engineering (CACRE), Dalian, China, 15–17 July 2021; pp. 414–422.
27. Bai, S.; Chen, F.; Englot, B. Toward autonomous mapping and exploration for mobile robots through deep supervised learning. In Proceedings of the 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Vancouver, BC, Canada, 24–28 September 2017; pp. 2379–2384.
28. Niroui, F.; Zhang, K.; Kashino, Z.; Nejat, G. Deep reinforcement learning robot for search and rescue applications: Exploration in unknown cluttered environments. *IEEE Robot. Autom. Lett.* **2019**, *4*, 610–617. [[CrossRef](#)]
29. Gkouletsos, D.; Iannelli, A.; de Badyn, M.H.; Lygeros, J. Decentralized Trajectory Optimization for Multi-Agent Ergodic Exploration. *IEEE Robot. Autom. Lett.* **2021**, *6*, 6329–6336. [[CrossRef](#)]
30. Garaffa, L.C.; Basso, M.; Konzen, A.A.; de Freitas, E.P. Reinforcement learning for mobile robotics exploration: A survey. *IEEE Trans. Neural Netw. Learn. Syst.* **2021**, 1–15. [[CrossRef](#)]
31. Yamauchi, B. A frontier-based approach for autonomous exploration. In Proceedings of the Proceedings 1997 IEEE International Symposium on Computational Intelligence in Robotics and Automation CIRA'97. 'Towards New Computational Principles for Robotics and Automation', Monterey, CA, USA, 10–11 July 1997; pp. 146–151.
32. Wu, C.-Y.; Lin, H.-Y. Autonomous mobile robot exploration in unknown indoor environments based on rapidly-exploring random tree. In Proceedings of the 2019 IEEE International Conference on Industrial Technology (ICIT), Melbourne, VIC, Australia, 13–15 February 2019; pp. 1345–1350.
33. Dang, T.; Khatkhat, S.; Mascarich, F.; Alexis, K. Explore locally, plan globally: A path planning framework for autonomous robotic exploration in subterranean environments. In Proceedings of the 2019 19th International Conference on Advanced Robotics (ICAR), Belo Horizonte, Brazil, 2–6 December 2019; pp. 9–16.

34. Da Silva Lubanco, D.L.; Pichler-Scheder, M.; Schlechter, T.; Scherhäufl, M.; Kastl, C. A review of utility and cost functions used in frontier-based exploration algorithms. In Proceedings of the 2020 5th International Conference on Robotics and Automation Engineering (ICRAE), Singapore, 20–22 November 2020; pp. 187–191.
35. Selin, M.; Tiger, M.; Duberg, D.; Heintz, F.; Jensfelt, P. Efficient autonomous exploration planning of large-scale 3-d environments. *IEEE Robot. Autom. Lett.* **2019**, *4*, 1699–1706. [\[CrossRef\]](#)
36. Bourgault, F.; Makarenko, A.A.; Williams, S.B.; Grocholsky, B.; Durrant-Whyte, H.F. Information based adaptive robotic exploration. In Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems, Lausanne, Switzerland, 30 September–4 October 2002; pp. 540–545.
37. Ila, V.; Porta, J.M.; Andrade-Cetto, J. Information-based compact pose SLAM. *IEEE Trans. Robot.* **2009**, *26*, 78–93. [\[CrossRef\]](#)
38. Julian, B.J.; Karaman, S.; Rus, D. On mutual information-based control of range sensing robots for mapping applications. *Int. J. Robot. Res.* **2014**, *33*, 1375–1392. [\[CrossRef\]](#)
39. Bai, S.; Wang, J.; Chen, F.; Englot, B. Information-theoretic exploration with Bayesian optimization. In Proceedings of the 2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Daejeon, Republic of Korea, 9–14 October 2016; pp. 1816–1822.
40. Bai, S.; Wang, J.; Doherty, K.; Englot, B. Inference-enabled information-theoretic exploration of continuous action spaces. In *Robotics Research*; Springer: Berlin/Heidelberg, Germany, 2018; pp. 419–433.
41. Wang, H.; Cheng, Y.; Liu, N.; Zhao, Y.; Cheung-Wai Chan, J.; Li, Z. An Illumination-Invariant Shadow-Based Scene Matching Navigation Approach in Low-Altitude Flight. *Remote Sens.* **2022**, *14*, 3869. [\[CrossRef\]](#)
42. Cai, C.; Chen, J.; Yan, Q.; Liu, F. A Multi-Robot Coverage Path Planning Method for Maritime Search and Rescue Using Multiple AUVs. *Remote Sens.* **2022**, *15*, 93. [\[CrossRef\]](#)
43. Shao, K.; Tang, Z.; Zhu, Y.; Li, N.; Zhao, D. A survey of deep reinforcement learning in video games. *arXiv* **2019**, arXiv:1912.10944.
44. Juliá, M.; Gil, A.; Reinoso, O. A comparison of path planning strategies for autonomous exploration and mapping of unknown environments. *Auton. Robots* **2012**, *33*, 427–444. [\[CrossRef\]](#)
45. Tai, L.; Liu, M. Mobile robots exploration through cnn-based reinforcement learning. *Robot. Biomim.* **2016**, *3*, 24. [\[CrossRef\]](#)
46. Dooraki, A.R.; Lee, D.J. Memory-based reinforcement learning algorithm for autonomous exploration in unknown environment. *Int. J. Adv. Robot. Syst.* **2018**, *15*, 1729881418775849. [\[CrossRef\]](#)
47. Gardner, J.R.; Kusner, M.J.; Xu, Z.E.; Weinberger, K.Q.; Cunningham, J.P. Bayesian optimization with inequality constraints. In Proceedings of the ICML, Beijing, China, 21–26 June 2014; pp. 937–945.
48. Feng, J.; Zhang, Y.; Gao, S.; Wang, Z.; Wang, X.; Chen, B.; Liu, Y.; Zhou, C.; Zhao, Z. Statistical Analysis of SF Occurrence in Middle and Low Latitudes Using Bayesian Network Automatic Identification. *Remote Sens.* **2023**, *15*, 1108. [\[CrossRef\]](#)
49. Renardy, M.; Joslyn, L.R.; Millar, J.A.; Kirschner, D.E. To Sobol or not to Sobol? The effects of sampling schemes in systems biology applications. *Math. Biosci.* **2021**, *337*, 108593. [\[CrossRef\]](#)
50. Schulz, E.; Speekenbrink, M.; Krause, A. A tutorial on Gaussian process regression: Modelling, exploring, and exploiting functions. *J. Math. Psychol.* **2018**, *85*, 1–16. [\[CrossRef\]](#)
51. Croci, M.; Impollonia, G.; Meroni, M.; Amaducci, S. Dynamic Maize Yield Predictions Using Machine Learning on Multi-Source Data. *Remote Sens.* **2022**, *15*, 100. [\[CrossRef\]](#)
52. Deng, D.; Duan, R.; Liu, J.; Sheng, K.; Shimada, K. Robotic exploration of unknown 2d environment using a frontier-based automatic-differentiable information gain measure. In Proceedings of the 2020 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM), Boston, MA, USA, 6–9 July 2020; pp. 1497–1503.
53. Shrestha, R.; Tian, F.-P.; Feng, W.; Tan, P.; Vaughan, R. Learned map prediction for enhanced mobile robot exploration. In Proceedings of the 2019 International Conference on Robotics and Automation (ICRA), Montreal, QC, Canada, 20–24 May 2019; pp. 1197–1204.
54. Zhelo, O.; Zhang, J.; Tai, L.; Liu, M.; Burgard, W. Curiosity-driven exploration for mapless navigation with deep reinforcement learning. *arXiv* **2018**, arXiv:1804.00456.
55. Shi, H.; Shi, L.; Xu, M.; Hwang, K.-S. End-to-end navigation strategy with deep reinforcement learning for mobile robots. *IEEE Trans. Ind. Inform.* **2019**, *16*, 2393–2402. [\[CrossRef\]](#)
56. Mnih, V.; Badia, A.P.; Mirza, M.; Graves, A.; Lillicrap, T.; Harley, T.; Silver, D.; Kavukcuoglu, K. Asynchronous methods for deep reinforcement learning. In Proceedings of the International Conference on Machine Learning, New York, NY, USA, 19–24 June 2016; pp. 1928–1937.
57. Li, T.; Ho, D.; Li, C.; Zhu, D.; Wang, C.; Meng, M.Q.-H. Houseexpo: A large-scale 2d indoor layout dataset for learning-based algorithms on mobile robots. In Proceedings of the 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Las Vegas, NV, USA, 24 October 2020–24 January 2021; pp. 5839–5846.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.